A Lesson Learned

In most languages, there aren't many ways to bind components together. Maybe you inherit from a class or you delegate to an object. If you want to get fancy, then you can use a library that specializes in managing dependencies—or even an entire framework.

Now, see how the authors of Rails bound their framework's parts together. In the very beginning, they probably just `included` and `extended` modules. Later, they sprinkled their code with metaprogramming fairy dust, introducing the include-and-extend idiom. Still later, as Rails kept growing, that idiom started creaking around the edges—so they replaced include-and-extend with the metaprogramming-heavy `ActiveSupport::Concern`. They evolved their own dependencies management system, one step at a time.

Over the years, we've learned that software design is not a "get it right the first time" affair. This is especially true in a malleable language such as Ruby, where you can use metaprogramming to change something as fundamental as the way that modules interact. So here is the main lesson I gained from the story of `Concern`: *metaprogramming is not about being clever—it's about being flexible*.

When I write my code, I don't strive for a perfect design at the beginning, and I don't use complex metaprogramming spells before I need them. Instead, I try to keep my code simple, using the most obvious techniques that do the job. Maybe at some point my code gets tangled, or I spot some stubborn duplication. That's when I reach for sharper tools, such as metaprogramming.

This book is full of metaprogramming success stories, and `ActiveSupport::Concern` is yet another one of them. However, `Concern`'s complex code and mildly controversial nature hint at a darker side of metaprogramming. This will be the subject of the next chapter, where we'll look at the story of Rails' most infamous methods.